

# Programação para IA

Machine Learning Aplicado à Engenharia de  
Petróleo

Eduardo Paiva

CESFI/UDESC - Aula 5

Balneário Camboriú



- **Revisão: Visualização Avançada**
- **Introdução ao Machine Learning**
- **Classificação de Litofácies**
- **Regressão para Previsão de Produção**
- **Clustering de Reservatórios**
- **Exercício Prático Integrado**

Ao final desta aula, você será capaz de:

- Aplicar algoritmos de classificação para identificar litofácies
- Construir modelos de regressão para previsão de produção
- Usar clustering para caracterizar reservatórios
- Avaliar performance de modelos com métricas apropriadas
- Interpretar resultados em contexto petrofísico
- Implementar pipeline completo de ML

## Competências desenvolvidas:

- Plotly para visualizações interativas
- Dashboards executivos profissionais
- Mapas geoespaciais de campos
- Storytelling com dados petrofísicos

## Conexão com Machine Learning:

- Visualizações ajudam na análise exploratória
- Gráficos revelam padrões para ML
- Dashboards mostram resultados de modelos
- Interpretabilidade visual dos algoritmos

## Revolução em curso na indústria:

- **Dados abundantes:** Logs, sísmica, produção, sensores
- **Decisões críticas:** Perfuração, completação, produção
- **Custos elevados:** Erros custam milhões
- **Padrões complexos:** Difíceis de modelar analyticamente

## Aplicações transformadoras:

- Classificação automática de litofácies
- Previsão de produção de poços
- Otimização de parâmetros de perfuração
- Detecção de anomalias em equipamentos
- Caracterização de reservatórios

## Aprendizado Supervisionado

### Dados rotulados disponíveis

- **Classificação:** Litofácies, qualidade de reservatório
- **Regressão:** Produção, propriedades petrofísicas

## Aprendizado Não-Supervisionado

### Descobrir padrões ocultos

- **Clustering:** Agrupamento de zonas similares
- **Redução de dimensionalidade:** Análise de múltiplas variáveis

## Foco da Aula

Supervisionado (classificação + regressão) e clustering básico

## Pipeline padrão na indústria:

- 1 **Coleta e Preparação:** Dados de logs, produção, geologia
- 2 **Análise Exploratória:** Visualização e estatísticas
- 3 **Feature Engineering:** Criação de variáveis derivadas
- 4 **Divisão Train/Test:** Separar dados para validação
- 5 **Treinamento:** Ajustar modelo aos dados
- 6 **Validação:** Avaliar performance em dados não vistos
- 7 **Interpretação:** Entender resultados geologicamente
- 8 **Deploy:** Aplicar em operações reais

**Cada passo é crítico para o sucesso**

## Por que Scikit-learn domina ML em Python?

- **API consistente:** Mesma interface para todos algoritmos
- **Algoritmos robustos:** Implementações otimizadas
- **Pré-processamento:** Ferramentas completas de preparação
- **Métricas:** Avaliação abrangente de modelos
- **Documentação:** Exemplos práticos abundantes

## Padrão fit/predict:

- `modelo.fit(X_train, y_train)` - Treinar
- `modelo.predict(X_test)` - Prever
- `modelo.score(X_test, y_test)` - Avaliar

## Problema real na indústria:

- **Desafio:** Identificar tipos de rocha automaticamente
- **Entrada:** Logs petrofísicos (GR, RT, NPHI, RHOB)
- **Saída:** Classe (Arenito, Folhelho, Carbonato)
- **Benefício:** Interpretação consistente e rápida

## Algoritmos principais:

- **Logistic Regression:** Simples e interpretável
- **Random Forest:** Robusto a outliers
- **Support Vector Machine:** Excelente para dados complexos
- **Gradient Boosting:** Estado da arte em performance

# Exemplo: Classificação de Litofácies

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report
# Preparar dados
features = ['GR', 'RT', 'NPHI', 'RHOB']
X = df[features]
y = df['Litologia']
# Dividir dados
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)
# Treinar modelo
modelo = RandomForestClassifier(n_estimators=100)
modelo.fit(X_train, y_train)
# Avaliar
predicoes = modelo.predict(X_test)
print(classification_report(y_test, predicoes))
```

## Como avaliar performance:

| Métrica          | Significado na Petrofísica                           |
|------------------|------------------------------------------------------|
| Accuracy         | Percentual de acertos na identificação de litofácies |
| Precision        | Quando prediz "Arenito", quantos realmente são?      |
| Recall           | Dos arenitos reais, quantos o modelo encontrou?      |
| F1-Score         | Média harmônica entre precision e recall             |
| Confusion Matrix | Matriz mostrando erros por classe                    |

## Interpretação geológica:

- Alta *precision* em "Reservatório" evita perfurações desnecessárias
- Alto *recall* em "Reservatório" não perde oportunidades

## Problema crítico na indústria:

- **Desafio:** Prever produção antes da completação
- **Entrada:** Propriedades petrofísicas, geometria, pressão
- **Saída:** Produção inicial (bbl/dia)
- **Benefício:** Decisões de investimento fundamentadas

## Algoritmos utilizados:

- **Linear Regression:** Baseline interpretável
- **Polynomial Features:** Capturar não-linearidades
- **Random Forest:** Lidar com interações complexas
- **Gradient Boosting:** Máxima performance preditiva

# Exemplo: Previsão de Produção

```
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_absolute_error, r2_score

# Preparar dados
features = ['Porosidade_Media', 'Permeabilidade_mD',
            'Saturacao_Agua', 'Profundidade_Total']
X = df[features], y = df['Producao_Inicial_bpd']

# Dividir dados
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

# Treinar modelo
modelo = RandomForestRegressor(n_estimators=100)
modelo.fit(X_train, y_train)

# Avaliar
predicoes = modelo.predict(X_test)
mae = mean_absolute_error(y_test, predicoes)
r2 = r2_score(y_test, predicoes)
print(f'MAE: {mae:.1f} bbl/dia, R : {r2:.3f}')
```

## Avaliando modelos preditivos:

| Métrica              | Interpretação Prática                             |
|----------------------|---------------------------------------------------|
| MAE                  | Erro médio em bbl/dia - fácil de interpretar      |
| RMSE                 | Penaliza mais erros grandes - crítico em petróleo |
| R <sup>2</sup> Score | Percentual da variação explicada pelo modelo      |
| MAPE                 | Erro percentual - útil para diferentes escalas    |

## Contexto industrial:

- MAE maior que 100 bbl/dia é considerado excelente
- R<sup>2</sup> menor que 0.8 indica modelo muito útil para decisões
- Erros em poços de alta produção são mais custosos

**Quais variáveis mais influenciam as predições?**

- **Random Forest:** Importância baseada na redução de impureza
- **Gradient Boosting:** Importância baseada em splits
- **Correlation Analysis:** Correlação linear simples

**Insights típicos na petrofísica:**

- ① **Permeabilidade:** Geralmente a mais importante
- ② **Porosidade:** Segunda em importância
- ③ **Saturação de água:** Crítica para produtividade
- ④ **Profundidade:** Relacionada à pressão

**Validação geológica sempre necessária!**

## Descobrir padrões ocultos:

- **Objetivo:** Agrupar zonas com propriedades similares
- **Entrada:** Múltiplas propriedades petrofísicas
- **Saída:** Grupos naturais de reservatórios
- **Benefício:** Segmentação para estratégias diferenciadas

## Algoritmos principais:

- **K-Means:** Rápido, número fixo de clusters
- **Hierarchical Clustering:** Visualização em dendrograma
- **DBSCAN:** Detecta clusters de densidade variável

**Aplicação:** Identificar "sweet spots" em reservatórios não convencionais

# Exemplo: Clustering de Reservatórios

```
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
# Preparar dados
features = ['Porosidade_Media', 'Permeabilidade_mD', 'Saturacao_Agua'], X = df[features]
# Normalizar (importante para K-means)
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
# Aplicar clustering
kmeans = KMeans(n_clusters=4, random_state=42)
clusters = kmeans.fit_predict(X_scaled)
# Adicionar ao dataframe
df['Cluster'] = clusters
# Analisar clusters
cluster_summary = df.groupby('Cluster')[features].mean(), print(cluster_summary)
```

## Evitando armadilhas comuns:

- **Overfitting:** Modelo decora dados, não generaliza (superestima)
- **Data leakage:** Usar informação do futuro (vazamentos)
- **Bias de seleção:** Dados não representativos (BIAS : enviesado)
- **Escala incorreta:** Não normalizar features

## Soluções robustas:

- **Cross-validation:** Validação cruzada k-fold
- **Hold-out set:** Conjunto separado para teste final
- **Feature scaling:** Normalização apropriada
- **Validação temporal:** Para dados de produção

## Regra de Ouro

Nunca usar dados de teste para ajustar o modelo!

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import RandomForestClassifier
# Pipeline integrado
pipeline = Pipeline([
    ('scaler', StandardScaler()),
    ('classifier', RandomForestClassifier(n_estimators=100))
])
# Treinar pipeline completo
pipeline.fit(X_train, y_train)
# Validacao cruzada
from sklearn.model_selection import cross_val_score
scores = cross_val_score(pipeline, X_train, y_train, cv=5)
print(f'CV Score: {scores.mean():.3f}    {scores.std():.3f}')
# Aplicar em novos dados
novas_predicoes = pipeline.predict(X_novos)
```

**Objetivo:** Implementar pipeline completo de classificação de litofácies

## Tarefas

- 1 Carregar dados dos poços da Bacia de Santos
- 2 Fazer análise exploratória das features
- 3 Dividir dados em treino/validação/teste
- 4 Treinar múltiplos algoritmos de classificação
- 5 Comparar performance usando métricas apropriadas
- 6 Analisar feature importance
- 7 Fazer previsões em novos dados

## Tempo Estimado

30 minutos - Trabalho em duplas com suporte do professor

## Exploração

- Classificação automática de fácies em dados sísmicos
- Previsão de sucesso de poços exploratórios
- Otimização de localização de poços

## Desenvolvimento

- Previsão de produção inicial de poços
- Otimização de parâmetros de completação
- Identificação de sweet spots em reservatórios

## Produção

- Previsão de declínio de produção; Detecção de anomalias em equipamentos; Otimização de injeção de água/gás

## ML não é solução mágica:

- **Qualidade dos dados:** Garbage in, garbage out
- **Interpretabilidade:** Nem sempre é clara
- **Generalização:** Pode não funcionar em novos campos
- **Manutenção:** Modelos degradam com o tempo

## Abordagem responsável:

- Sempre validar geologicamente
- Começar com modelos simples e interpretáveis
- Manter expertise técnica no processo
- Monitorar performance continuamente
- Documentar suposições e limitações

**ML complementa, não substitui, expertise geológica**

## Tendências emergentes:

- **Deep Learning:** Redes neurais para problemas complexos
- **Transfer Learning:** Aproveitar modelos de outros campos
- **AutoML:** Automatização da criação de modelos
- **Explainable AI:** IA mais interpretável
- **Edge Computing:** ML em tempo real no campo

## Competências do futuro:

- Combinação de expertise geológica + ML
- Pensamento estatístico rigoroso
- Capacidade de validar modelos criticamente
- Comunicação de incertezas para tomadores de decisão

## Aula 6 - Deep Learning e Redes Neurais:

- Fundamentos de redes neurais
- Aplicações em dados sísmicos
- Redes convolucionais para imagens de reservatório
- Redes recorrentes para séries temporais de produção

## Preparação recomendada:

- Revisar conceitos de ML desta aula
- Praticar com biblioteca TensorFlow/Keras
- Explorar exemplos de deep learning em geociências

**Do aprendizado clássico às redes neurais profundas!**

## Projeto: Modelo Preditivo Personalizado

### 1 Escolher problema:

- Classificação de qualidade de reservatório; Previsão de produção de poços; Clustering de zonas produtivas

### 2 Desenvolver solução completa:

- Pipeline de dados limpo
- Comparação de múltiplos algoritmos
- Validação rigorosa com métricas apropriadas
- Interpretação geológica dos resultados

### 3 Apresentar em 5 minutos:

- Problema e dados utilizados
- Metodologia e algoritmos testados
- Resultados e interpretação técnica
- Recomendações para aplicação prática

**Entrega:** Antes da próxima aula (Via Moodle)

**Eduardo Paiva**

**Email:** [eduardo.paiva@udesc.br](mailto:eduardo.paiva@udesc.br)

**Sala:** 38

**Atendimento:** Quintas-feiras, 13:30 às 15:00h

**Próxima aula: Deep Learning e Redes Neurais!**

**Obrigado!**